

Elevator Math Modeling Solution

Harrison, Daniel, Connor

August 2024

Contents

1	Memo #4	3
2	Assumptions	4
3	Model Description	4
3.1	Time Step Updates	5
3.1.1	Employee Arrival	5
3.1.2	Elevator Updates	5
3.1.3	Strategy Function	5
3.2	Model of the Current Situation	5
3.3	Model of the Situation After Implementation of Changes	6
4	Appendix 1: GitHub Link	7

1 Memo #4

From: Connor, Daniel, & Harrison
To: Boss
Re: Solutions to the lateness problem

Hello Boss,

To fix this problem, we began by reaching out to our assistant for further details on the elevator system. We discovered that the elevators take 5 seconds between each floor, 10 seconds for each stop, 8 seconds if the doors have to reopen, and 15 seconds for the elevators to fill on the ground floor. We assumed that it takes approximately 1.5 seconds for each person to enter the elevator. Additionally, there are 4 elevators each with a 10 person max capacity, and about 110 people are arriving late.

The problem right now is that our employees are not using the elevators efficiently. As workers arrive in the lobby, every elevator is packed with workers from a wide assortment of floors in a first come, first serve approach. This is human nature: each person prioritizes the efficiency of their own arrival. However, the randomness of each person in any given elevator drastically increases the amount of time it takes for all employees to make it to their intended destination.

To model the problem and test solutions, we created a model of the problem in Python, a programming language. We assumed that people were arriving in a normal distribution between 8:35-9:00 such that the random strategy would make around 110 people be late to work.

We experimented with many systems, and all of them were much more efficient than the current one. We picked the simplest one to implement, as they are all very similar in performance. This system requires employees to line up by the floor they need to go to when they arrive in the lobby. When an elevator opens, we first check how many workers are waiting in the lobby for each floor, and take as many people as possible from the floor with the most workers waiting. If this floor has more than ten people, then our elevator only takes ten people—the ten people in this group who arrived earliest. Otherwise, the elevator takes everyone on this floor.

We also want to maximize the number of people in our elevators to get the most use possible out of our trips. Say the floor with the most people waiting in the lobby to get there is Floor X . In our proposed solution, we check Floor $X - 1$ to see if any workers need to reach this floor who are currently waiting in the lobby. If there exist such workers, we take as many of them as we can fit. If our elevator is full, the elevator leaves to drop its passengers off at their respective floors. Otherwise, we repeat the same process with Floor $X - 2$, and we keep repeating this until we reach Floor 1. At this point, if the elevator still is not at its maximum capacity, we repeat this process for Floor $X + 1$, and then Floor $X + 2$, all the way to Floor 6. Even if the elevator is not at its maximum capacity at this point (i.e. there are less than ten people currently waiting in the lobby), the elevator leaves. At any point if the elevator reaches its maximum capacity while looping through the floors in this specific way, the elevator leaves.

The efficiency of this proposed solution is expressed in more detail in the following pages. Given that we were not working with complete information on the situation, we were required to make some assumptions to gain a better understanding of the effect our proposed solution has on workers' efficiency, which is also explained later on. We hope our model mitigates worker tardiness and we would be happy to work on similar problems in the future.

2 Assumptions

1. All 500 employees will come in normal distribution randomly according to a random distribution.
2. The elevator doors only open once every trip because all workers are capable of pressing the correct floor level button and departing at the right floor consistently.
3. All employees use the elevators to go to their floor directly. They do not leave the ground lobby once they arrive, and they do not leave their floor level after they arrive there.
4. It takes 1.5 seconds to load each person into an elevator in the lobby, 10 seconds every time the door opens, and 5 seconds for each floor the elevator passes.
5. An average of 110 people are clocking in after 9:00 AM.
6. There are 4 elevators, and each of them have a maximum capacity of 10 people.
7. Employees are counted as arriving as soon as they step off the elevator.

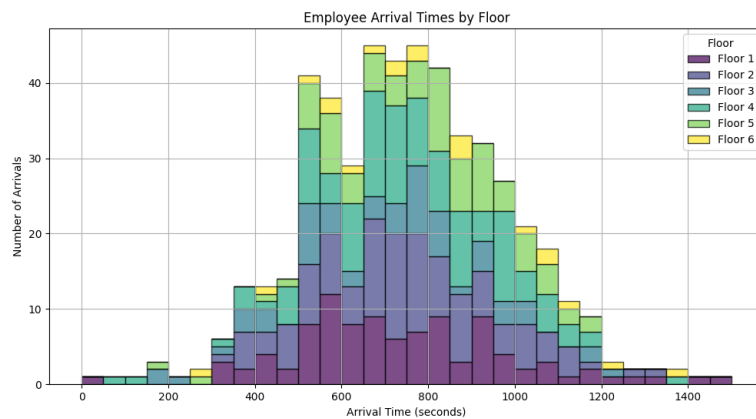
3 Model Description

Our model is implemented in the Python programming language and is based on a simulation loop with time steps. It takes an input of a strategy function, and simulates a morning rush hour using that strategy, resulting in the amount of time it took to send all of the employees to their floors, how many employees were late, and a list of each employee along with what time they were able to clock in by.

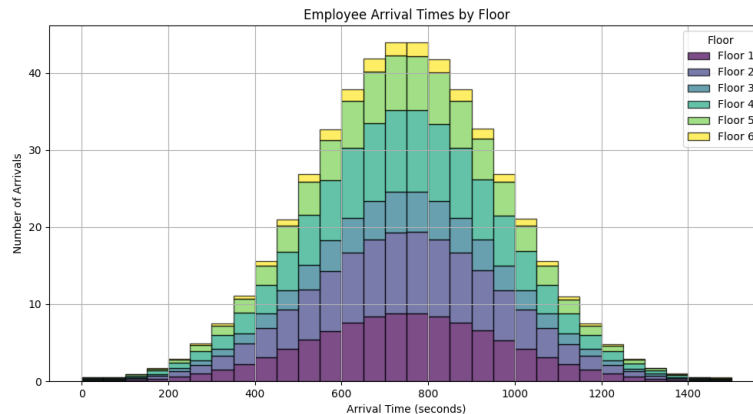
This strategy function would take an input of the current state of the lobby and the current state of the elevators, and return a list of 4 elements, each being a list of 6 numbers representing how many employees of the corresponding floor to take.

To start, it generates the arrival times according to the normal distribution and pairs them with floor numbers. The parameters in the normal distribution were derived from the fact that 110 people were late. These parameters are a mean of 750 and a standard deviation of 225 seconds where arrival times before 0 and 1500 are clipped to 0 and 1500. In the model, the start of the simulation seconds is defined as 8:35 in real life.

Here is a histogram of a single generated arrival times:



Here is a histogram of the average of 10000 generated arrival times:



3.1 Time Step Updates

Then, we enter the simulation loop where a variable called "simulation seconds" that starts at 0 is incremented to keep track of the time that has passed. There are three big parts to the simulation loop:

3.1.1 Employee Arrival

We start by checking if any employees arrived by iterating through the arrival times and seeing if they are less than the simulation seconds. If they did, we move them to a list called the lobby and remove them from their original list.

3.1.2 Elevator Updates

Then, the elevators are updated. If they are not available, the time it takes for them to be available is decremented. Additionally, in the elevator class, there is a list of what each passenger is taking and how long it will take those passengers to get to their floors. Thus, if the elevator has any passengers, the time it takes for them to get to their floors is also decremented. If they arrive at their floor, a list will be updated to record that.

3.1.3 Strategy Function

Lastly, we used the strategy function to determine which people in the lobby should go in which available elevators (if there are any). If any elevators are used this way, they are updated and a list of passengers and their arrival times is generated, along with the time it would take for the elevator to get back to the lobby. This information is stored on each elevator.

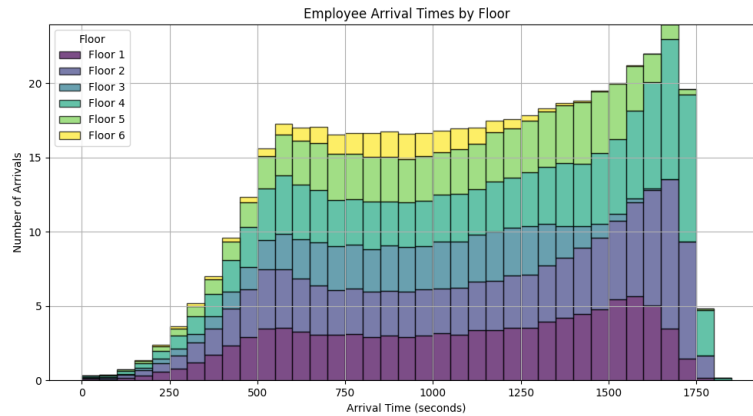
When the list containing the arrival times of the employees is empty, we end the simulation, with the final time being calculated as the total number of time steps plus the time it will take the last employees to get to their floors. This is returned along with information about late employees that was collected during the simulation.

Additionally, when the time step is equal to the time corresponding to 9:00 AM (1500 seconds), the list containing information about people who have already arrived is copied and returned along with all the other information at the end of the simulation.

3.2 Model of the Current Situation

The problem right now is that our employees are not using the elevators efficiently. As workers arrive in the lobby, every elevator is packed with workers from a wide assortment of floors in a first come, first serve approach.

Histogram of the average times it takes employees to arrive to their floors current situation over 1000 iterations.



As you can see, many people are arriving too late (1500 is 9:00 AM). On average over 1000 iterations, using this inefficient strategy, employees were able to arrive at their floor by 9:04 and an average of 111.043 people were late.

3.3 Model of the Situation After Implementation of Changes

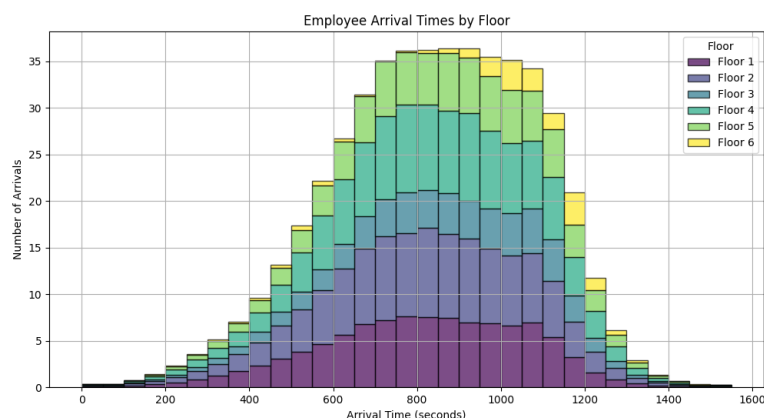
We modeled many different strategies and they were all much more efficient than the current one. We picked the proximity system due to its simplicity and strong performance.

The proximity system first identifies the floor with the most people—let's call this floor X —and fits as many people from this floor as possible into the elevator. Then, we check the floor directly below floor X to see if there are any workers currently waiting in the lobby who wish to travel to floor $X - 1$. If so, we repeat our aforementioned algorithm: we fit as many people from floor $X - 1$ into the elevator along with everyone from floor X , and if our elevator is at maximum capacity it leaves to drop off its workers. If our elevator is not at maximum capacity, we check floor $X - 2$, and the process continues.

We repeat this algorithm until our elevator has reached maximum capacity or until we have checked floor 1 (i.e. we checked all floors from floor 1 to floor X). If we have checked floor 1 and our elevator is still not at maximum capacity, we then check floor $X + 1$ in this similar manner, and we work our way upwards until we have reached floor 6. At this point we have checked all floors 1-6, and regardless of whether we are at maximum capacity or not, we send the elevator to drop its workers off.

As you can see, barely any people are arriving late, and the curve of arrivals is much smoother. On average over 1000 iterations, using this strategy, all workers arrived to their floors by 8:59 with 0.237 late employees.

Histogram of the average times it takes employees to arrive to their floors using the proximity system over 1000 iterations.



4 Appendix 1: GitHub Link

<https://github.com/Moonflower2022/elevator-problem>